

计算机组成原理

July 15, 2026

1	计算机系统概述	3
1.1	计算机的性能指标	3
2	数据的表示和运算	4
2.1	定点数的编码表示	4
2.2	整数的表示	5
2.2.1	C 语言中的整型数据类型	5
2.2.2	不同字长整数之间的转换	5
2.3	运算方法	5
2.3.1	移位运算	5
2.3.2	标志位	6
2.4	浮点数的表示	6
2.4.1	理论浮点数	6
2.4.2	IEEE 754 标准	7
2.4.3	加减运算	7
2.4.4	C 语言中的浮点数类型	8
3	存储系统	8
3.1	存储器的性能指标	8
3.2	RAM (Random Access Memory, 随机存储器)	9
3.2.1	DRAM	9
3.3	多模块存储器	10
3.4	外部存储器	10
3.4.1	磁盘存储器	10
3.4.2	固态硬盘	11
3.5	高速缓冲存储器	11
3.5.1	Cache 与主存映射方式	11
3.5.2	Cache 一致性问题	12
4	指令系统	13
4.1	指令的寻址方式	14
4.1.1	指令寻址	14
4.1.2	数据寻址	14
4.2	程序的机器级代码表示	16
4.2.1	常用指令	16
4.3	CISC & RISC	17

5	总线	18
5.1	系统总线的结构	19
5.2	总线的性能指标	19
5.3	总线事务	20
5.4	总线定时	20
5.4.1	异步定时方式	21
6	输入/输出系统 & 管理	21
6.1	I/O 接口	21
6.1.1	I/O 接口的基本结构	22
6.1.2	I/O 接口的功能	22
6.2	I/O 控制方式	23
6.2.1	程序查询方式（程序直接控制方式、程序轮询方式）	23
6.2.2	程序中断（中断驱动方式）	23
6.2.3	DMA 方式（Direct Memory Access, 直接存储器存取）	25
6.2.4	计算	27
6.3	I/O 软件层次结构	27
6.3.1	逻辑设备名 VS 物理设备名	28
6.3.2	设备分配	28
6.4	应用程序 I/O 接口	29
6.5	缓冲管理	29
6.5.1	缓冲区（Buffer）	29
6.5.2	SPOOLing 技术（假脱机技术）	31

1 计算机系统概述

名称	英文	含义
存储元	Memory Cell	一个二进制位 (1 bit)
存储单元	Storage Unit	一个地址对应的数据
存储字	Memory Word	一个存储单元存放的数据 (CPU 一次存取的数据单位、存储单元存储的一串二进制代码)
存储字长	Memory Word Length	一个存储单元包含的二进制位数 (由编址方式决定)
机器字长	Machine Word Length	CPU 一次能够自然处理的二进制数据位数 (由 CPU 决定)

1. 按地址存取方式：(主存储器的工作方式是) 按存储单元的地址进行存取
2. MAR (存储器地址寄存器, Memory Address Register)：存放访存地址，位数反映最多可寻址的存储单元的个数，位数与地址线位数相同
 - 地址个数 = 终止地址 - 起始地址 + 1
 - 地址位数 = $\log_2$ 存储单元个数
 - ① 字节编址 (存储字长 1B = 8bit)：存储单元个数 = 存储器容量/1B。e.g. 主存容量 32MB，字节编址 $\Rightarrow$ 地址位数 = $2^5 * 2^{20}$
 - ② 32 位字编址 (存储字长 32bit)：存储单元个数 = 存储器容量/32bit。e.g. 主存容量 32MB，字编址 $\Rightarrow$ 地址位数 = 2^{20}
3. MDR (存储器数据寄存器, Memory Data Register)：用于暂存要从存储器中读或写的信息，位数与数据总线宽度 (位数) 相同

Example

某 64 位计算机，主存容量 4MB，字节编址

$$\Rightarrow \begin{cases} \text{机器字长} = 64\text{bit} \\ \text{存储字长} = 1\text{B} = 8\text{bit} \\ \text{MDR 位数} = 64\text{bit} \\ \text{MAR 位数} = \log_2 \frac{4\text{MB}}{1\text{B}} = 2^{22} \end{cases}$$

4. CPU 一次从主存读/写多少数据，取决于：
 - 数据总线宽度
 - 存储器接口设计
 - 一次访存操作的宽度
 - 数据总线：64bit，那么一次访存可以读：64bit = 8Byte。也就是一次可以读 8 个按字节编址的存储单元

1.1 计算机的性能指标

1. CPU 时钟周期：CPU 工作的最小时间单位
2. 主频 (CPU 时钟频率)：每秒有多少个时钟周期，单位 Hz

$$\text{主频} = \frac{1}{\text{CPU 时钟周期}}$$

3. CPI (CyclePerInstruction): 执行一条指令所需的时钟周期数

4. IPS (Instruction Per Second): 每秒执行多少条指令

$$IPS = \frac{\text{主频}}{\text{平均CPI}}$$

5. CPU 执行时间: 运行一个程序所花费的时间

$$CPU\text{执行时间} = \frac{CPU\text{时钟周期数}}{\text{主频}} = \frac{\text{指令条数} \times CPI}{\text{主频}}$$

6. MIPS (Million Instruction Per Second): 每秒执行多少百万条指令

$$MIPS = \frac{\text{指令条数}}{\text{执行时间} \times 10^6} = \frac{\text{主频}}{CPI * 10^6}$$

7. 机器的速度与基准程序在该机器上的运行时间呈相反关系

2 数据的表示和运算

2.1 定点数的编码表示

1. 正数: 原码 == 反码 == 补码

2. 负数

- 原码 $\Leftrightarrow$ 反码

- ① 符号位保留, 按位取反, 末位加 1

- ② 符号位保留, 最右 1 及右边保留, 左边取反

- $[x]_{\text{补}} \Rightarrow [-x]_{\text{补}}$

- ① 按位取反, 末位加 1

- ② 最右 1 及右边保留, 左边取反

3. 移码: 将补码的符号位取反

4. 机器数 $\Rightarrow$ 真值

- 权值法

- ① 8 位补码: $11110110 \Rightarrow -2^7 + 2^6 + 2^5 + 2^4 + 2^2 + 2^1 = -10$

- ② 16 位补码: $8009H = 1000000000001001 \Rightarrow -2^{15} + 2^3 + 2^0 = -32759$

- ③ 32 位补码: $FFFF8009H$ 前面全是 F , 说明: $8009H$ 被符号扩展到了 32 位, 所以真值和 $8009H$ 相同

- 关键数

- ① $8000H = -32768, 8001H = -32767, 8002H = -32766$

- ② $7FFFH = 32767$

- ③ $FFFFH = -1, FFFE H = -2, FFFCH = -4$

2.2 整数的表示

1. 计算机中的有符号整数都用**补码**表示，所以 n 位有符号整数的表示范围 $-2^{n-1} \sim 2^{n-1} - 1$

2.2.1 C 语言中的整型数据类型

1. *short* 或 *short int*: 16bit
2. 整型 *int*: 32bit, 真值范围: $[-2^{31} \sim 2^{31} - 1] = [-2.147483648 * 10^9 \sim 2.147483647 * 10^9]$
3. *long* 或 *long int*: 32 位机器中 32 位, 64 位机器中 64 位
4. 字符型 *char*: 8bit

2.2.2 不同字长整数之间的转换

1. 大字长 $\Rightarrow$ 小字长: 高位部分直接截断, 低位部分直接赋值
2. 小字长 $\Rightarrow$ 大字长: 高位扩展
 - 无符号整数: **零扩展**
 - 有符号整数: **符号扩展**。**符号扩展前后真值保持不变**

2.3 运算方法

1. 与 ($\cdot$), 或 ($+$), 非, 异或 ($\oplus$, 输入相异则为 1)
2. 加法: 补码直接相加。**溢出**: 正 + 正 = 负 (上溢)、负 + 负 = 正 (下溢)
3. 减法: 被减数与减数的负数补码相加。**溢出**: 负 - 正 = 正、正 - 负 = 负
4. 乘法: 左移一位乘以 2; 加法。**溢出判断**
 - 有符号数: 高 X 位的每一位都相同, 且都等于低 X 位的符号, 则表示**不溢出**, 否则表示溢出
 - 无符号数: 高 X 位全为 0, 表示**不溢出**, 否则表示溢出
5. 除法: 右移一位除以 2; 减法。
 - 溢出判断: 若是 $2n$ 位除以 n 位的无符号数, 商的位数为 $n + 1$ 位, 当第一次试商为**1**时, 则表示结果**溢出**
 - 若是两个 32 位 *int* 型整数相除, 则除了 $\frac{\text{绝对值最大的负数}}{-1}$ 会溢出, 其余情况都不会溢出
6. 无符号数和有符号数一起参与运算时, 计算机按**无符号数**来解释最终的执行结果

2.3.1 移位运算

1. 逻辑移位 $\Rightarrow$ 无符号整数
 - 左移高位移出, 低位补 0
 - 右移低位移出, 高位补 0
 - **溢出**: 高位 1 移出, 发生溢出
 - **丢失精度**: 低位 1 移出, 丢失精度
2. 算术移位 $\Rightarrow$ 有符号整数

- 左移高位移出，低位补 0
- 右移低位移出，高位补符号位
- 溢出：左移前后的符号位不同，发生溢出
- 丢失精度：低位 1 移出，丢失精度

2.3.2 标志位

1. 零标志 ZF: $ZF = 1$ 表示结果为 0
2. 溢出标志 OF: 判断有符号数, $OF = C_n \oplus C_{n-1}$ (符号位进位与最高位进位的异或结果)
3. 符号标志 SF: 表示结果的符号
4. 进/错位标志 CF: 表示无符号数运算时的进位/借位
 - 加法: $CF = 1$ 结果溢出。最高位是否产生进位
 - 减法: $CF = 1$ 表示有借位，即不够减

2.4 浮点数的表示

2.4.1 理论浮点数

1. 原码尾数规格化
 - 正数: $0.1xxx$
 - 负数: $1.1xxx$
2. 补码尾数规格化: 符号位与最高数值位必须相反。
 - $+0.75 \Rightarrow 0.11$
 - $-0.75 \Rightarrow 1.01$

2.4.2 IEEE 754 标准

		单精度	双精度
符号 s		1bit	1bit
阶码 e		8bit(范围: [1, 254])	11bit(范围: [1, 2046])
尾数 f (尾数有隐藏位 1)		23bit	52bit
总位数		32bit (原码)	64bit (原码)
偏置值		127(7FH)	1023(3FFH)
指数		阶码真值 + 偏置值, 后按无符号整数规则转换	
真值		$(-1)^s \times 1.f \times 2^{e-127}$	$(-1)^s \times 1.f \times 2^{e-1023}$
最小值		$e = 1, f = 0 \Rightarrow 1.0 * 2^{1-127} = 2^{-126}$	$e = 1, f = 0 \Rightarrow 1.0 * 2^{1-1023} = 2^{-1022}$
最大值		$e = 254, f = .111 \dots \Rightarrow 1.111 \dots 1 * 2^{254-127} = 2^{127} * (2 - 2^{-23})$	$e = 2046, f = .111 \dots \Rightarrow 1.111 \dots 1 * 2^{2046-1023} = 2^{1023} * (2 - 2^{-52})$
阶码全 1	尾数全 0	$\pm\infty$	
	尾数非 0	NaN	
阶码全 0	尾数全 0	± 0	
	尾数非 0	非规格化数 (隐藏位为 0)	
判断溢出		规格化后阶码超出所能表示的范围时, 发生溢出	

1. 真值 $\Rightarrow$ IEEE 754 浮点数

- ① 确定符号位
- ② 真值转化为二进制
- ③ 规格化确定尾数和阶码真值
 - 左规: 尾数每左移一位, 阶码减 1
 - 右规: 尾数每右移一位, 阶码加 1
- ④ 阶码真值 + 偏置值, 后按无符号整数规则转换

2.4.3 加减运算

1. 对阶: 使两个操作数的小数点对齐

- 小阶码向大阶码看齐
- IEEE 规定右移出的 *bit* 至少额外保留 3 位

2. 尾数加减

3. 尾数规格化

4. 舍入

- 就近舍入：0 舍 1 入。特殊情况：3 个舍弃位是 100
 - ① 1 + 23bit 尾数末位 0 $\Rightarrow$ 截断
 - ② 1 + 23bit 尾数末位 1 $\Rightarrow$ 截断多余位，并末位加 1
- 场景
 - ① 对阶
 - ② 右规格化

5. 溢出判断

- 一个正指数超过了最大允许值 (127 或 1023) $\Rightarrow$ 指数上溢 (阶码 11111111)，将尾数强制保存为全 0
- 一个负指数超过了最小允许值 (-149 或 -1074) $\Rightarrow$ 指数下溢 (若当前阶码 00000001，尾数仍需左规，直接将阶码置为全 0，并且尾数不移位)

2.4.4 C 语言中的浮点数类型

1. 不同类型数的混合运算，遵循的原则是类型提升，即较低类型转换为较高类型
2. *float* : 4Byte, 真值范围: $[-3.4 * 10^{38} \sim 3.4 * 10^{38}]$
3. *double* : 8Byte, 真值范围: $[-1.79 * 10^{308} \sim 1.79 * 10^{308}]$
4. *int* $\Rightarrow$ *float*: 可能精度丢失
5. *double* $\Rightarrow$ *float*: 可能溢出或精度丢失
6. *float/double* $\Rightarrow$ *int*: 仅保留整数部分，可能溢出

3 存储系统

3.1 存储器的性能指标

1. 存储容量 = 存储字数 $\times$ 存储字长
 - 存储字数：表示存储器可编址的存储单元个数 (即地址空间大小)，决定所需地址位数
 - ① 4M 的存储字数：即一共有 $4M = 4 * 2^{20} = 2^{22}$ 个存储单元，可使用引脚复用技术，分为行、列引脚，共 11 个引脚，先后两次输入，需要 11 根地址引脚
 - 存储字长：表示一个存储单元包含的数据位数，即每个存储单元的比特数据，也表示数据引脚的数量
 - ① 按字节编址：存储容量 $4MB = 4M \times 8bit$ 。存储字长 8bit，即一个地址对应 8bit 数据，8 根数据引脚

② 按字编址：存储容量 $16MB = 4M \times 32bit$

③ 按半字编址：存储容量 $8MB = 4M \times 16bit$

2. 单位成本：位价 = 总成本/总容量

3. 存储速度：数据传输速率，即每秒传送信息的位数。存储速度 = 存储字长/存取周期 = 存取时间 + 恢复时间

- 存取时间 (T_a)：从启动一次存储器操作到完成该操作所经历的时间
- 存取周期 (T_m)：存储器进行一次完整的读/写操作所需的全部时间，即连续两次独立访问存储器操作之间所需的最小时间间隔
- 主存带宽 (B_m , 数据传输速率)：表示每秒从主存进出信息的最大数量

3.2 RAM (Random Access Memory, 随机存储器)

RAM = SRAM + DRAM

1. SRAM (Static Random Access Memory, 静态随机存储器)：即使信息被读出后，它仍保持器原状态而不需要再生 (非破坏性读出)
2. DRAM (Dynamic Random Access Memory, 动态随机存储器)：Cache。必须定时刷新和读后再生
3. 主存由 RAM 和 ROM 构成

3.2.1 DRAM

1. 刷新周期：对同一行进行相邻两次刷新的时间间隔，通常取 $2ms$ 。e.g. $64K \times 1bit$ 的 DRAM 芯片构成 $128K \times 8bit$ 的存储器，若采用异步刷新方式，每单元存储间隔不超过 $2ms$ ，则生成的刷新信号的间隔时间是？若采用集中刷新方式，则存储器刷新一遍最少用多少个读/写周期？

解析

$64K \times 1bit$ 由一个 256×256 的为平面组成
构成存储器的所有芯片同时按行刷新，每个芯片有 256 行，所以存储器所有单元刷新一遍至少需要 256 次刷新操作
若采用异步刷新方式，则相邻两次刷新信息的时间间隔为 $2ms/245$
若采用集中刷新方式，则整个存储器刷新一遍至少需要 256 个读/写周期

2. DRAM 的刷新单位是行，逐行定时刷新，没刷新一行，需消耗一个存取周期
3. 每个刷新周期需刷新 $2^{\text{行地址数}}$ ，即若行地址 $6bit$ ，则每个刷新周期需刷新 $2^6 = 64$ 次，消耗 64 个存取周期
4. 再生仅需恢复被读出的那些单元的数据
5. 行缓冲器：缓存指定行中每列的数据，大小 = 列数 $\times$ 每列的数据位宽，常用 SRAM 实现，可支持突发传输
6. DRAM 存取周期 $>$ SRAM 存取周期 $\Rightarrow$ 一次突发传输决不能跨越 DRAM 行边界

3.3 多模块存储器

1. 单体多字存储器

2. 多体并行存储器

- 高位交叉编址

① 连续读取 m 个字所需的时间 $t = mT$

- 低位交叉编址：程序连续存放在相邻模块中。可采用轮流启动或同时启动方式

① 模块的存取周期为 T ，总线周期 r ，存储器交叉模块数应大于或等于 $m = \frac{T}{r}$

② 连续存取 m 个字所需的时间 $t = T + (m - 1)r$

3.4 外部存储器

3.4.1 磁盘存储器

1. 磁盘地址结构：驱动器号 - 柱面（磁道）号 - 盘面（磁头）号 - 扇区号

磁盘的性能指标

1. 记录密度：盘片单位面积上记录的二进制数据量

- 道密度：沿磁盘半径方向单位长度上的磁道数
- 位密度：磁道单位长度上能记录的二进制代码位数
- 面密度：位密度和道密度的乘积

2. 磁盘的容量

- 非格式化容量：磁记录编码可利用的磁化单位总数。非格式化容量 = 记录面数 × 柱面数 × 每条磁道的磁化单位数
- 格式化容量：按照某种特定的记录格式所能存储信息的总量。格式化容量 = 记录面数 × 柱面数 × 每道扇区数 × 每个扇区的容量
- 格式化后的容量比非格式化容量要小

3. ★存取时间 = 平均寻道时间 + 平均旋转延迟时间 + 传输时间

- 寻道时间：磁头移动到目的磁道的时间
- 平均寻道时间：从最外道移动到最内道时间的一半，和转速无关
- 旋转延迟时间：磁头定位到要读/写扇区的时间
- 平均旋转延迟时间：旋转半周的时间，和转速有关
- 传输时间：传输数据所花费的时间

4. 数据传输速率：在单位时间内向主机传送数据的字节数。假设磁盘转数为 r 转/秒，每条磁道容量为 N 字节，则数据传输速率为

$$D_r = rN = \frac{\text{每个磁道容量}}{\text{读一个磁道数据的时间}}$$

5. 读一个磁道数据的时间 = 磁盘旋转一周的时间 - 通过扇区间隙的总时间

3.4.2 固态硬盘

1. 数据以页为单位读/写，以块为单位擦除

3.5 高速缓冲存储器

1. 时间局部性：某个数据/指令在短时间内被反复访问
2. 空间局部性：最近的未来要用到的信息，很可能与现在正在使用的信息在存储空间上是临近的
3. CPU 与 Cache 之间的数据交换以字（机器字长）为单位
4. Cache 与主存之间的数据交换以 Cache 块为单位。Cache 块也称 Cache 行，每块由若干字节组成，块的长度称为块长，也称行长
5. Cache 命中率：CPU 与访问的信息已在 Cache 中的比率
6. Cache-主存系统的平均访问时间
 - t_c ：命中时的 Cache 访问时间
 - t_m ：未命中（缺失）时的访问时间
 - H ：命中率
 - $1 - H$ ：未命中率（缺失率）
 - 串行访问： $T_a = Ht_c + (1 - H)(t_c + t_m)$
 - 并行访问： $T_a = Ht_c + (1 - H)t_m$

3.5.1 Cache 与主存映射方式

1. 标记项：包含有效位、脏位（修改位）、替换算法位、标记位
2. 标记位：指明 Cache 块是主存中哪一块的副本
3. 有效位：说明 Cache 行中的信息是否有效
4. 地址映射：把主存地址空间映射到 Cache 地址空间，即把存放在主存中的信息按照某种规则装入 Cache
 - 直接映射：主存中的每一块只能装入 Cache 中的唯一位置
 - ① 块冲突：若这个位置已有内容，则产生块冲突，原来的块将无条件地被替换出去
 - ② 命中：根据访存地址中间的 Cache 行号位数，找到对应 Cache 行，将对应 Cache 行中的标记与主存地址的标记位数（高位）进行比较，若相等且有效位为 1，则命中
 - ③ Cache 不存 Cache 行号和块内地址
 - ④ 计算
 - (a) 主存容量 $\Rightarrow$ 主存地址位数：主存容量 = 存储字数 $\times$ 存储字长，主存容量 1MB，按字节编址，存储字数 = $1MB/1B = 2^{20}$ ，即主存地址 20bit
 - (b) Cache 有效容量：Cache 中存数据的容量，即数据块总大小
 - (c) Cache 行结构 = 有效位 + 脏位（修改位）+ 替换算法位 + Tag + 数据位（块大小 32B $\Rightarrow$ 数据位 $32 * 8\text{bit} = 256\text{bit}$ ）

- (d) Cache 容量 = (数据块 (行长) + Tag + 控制位) * 行数/块数: Cache 数据区大小 32KB, 主存块大小 32B, 行数 = $32KB/32B = 2^{10}$, 即 10bit 行号
- (e) 行长/块长 $\Rightarrow$ 块内地址位数: 每块 $16B = 2^4B$, 即块内地址 4bit
- (f) 行数/块数 $\Rightarrow$ 行号位数: Cache 行号位数 = $\log_2$ Cache 行数
- (g) 主存块号 = $\frac{\text{主存地址}}{\text{行长}}$
- (h) Cache 行号 = 主存块号 mod Cache 总行数
- (i) 主存地址结构: 标记位 - Cache 行号 - 块内地址 (与快长、行长有关)
- (j) 主存块号 = 标记位 + Cache 行号
- 全相联映射: 主存中的每一块可以装入 Cache 中的任何位置
 - ① 优点
 - (a) Cache 块的冲突概率低, 只要有空闲 Cache 行, 就不会发生冲突
 - (b) 空间利用率高
 - (c) 命中率高
 - ② 缺点
 - (a) 标记的比较速度较慢
 - (b) 实现成本较高, 通常需采用按内容寻址的相联存储器
 - ③ 主存地址结构: 标记 - 块内地址 (与快长、行长有关)
- 组相联映射: 将 Cache 分成 Q 个大小相等的组, 每个主存块可以固定组中的任意一行, 即组间采用直接映射, 组内采用全相联映射
 - ① 每组 Cache 行的数量越大, 发生块冲突的概率越低
 - ② r 路组相联: 每组中有 r 个 Cache 行
 - ③ Cache 组数 = $\frac{\text{Cache 总容量}}{\text{Cache 块大小} \times r}$
 - ④ Cache 组号 = 主存块号 mod Cache 组数 (Q)
 - ⑤ 主存地址结构: 标记 - 组号 - 块内地址 (与快长、行长有关)
 - ⑥ 主存块号 = 标记位 + 组号
 - ⑦ 命中: 根据访存地址中间的组号找到对应的 Cache 组, 将对应的 Cache 组中每个行的标记与主存地址的高位标记进行比较, 若相等且有效位 1, 则命中
- 计算

3.5.2 Cache 一致性问题

1. Cache 写操作命中

- 全写法 (直写法, Write Through): 当 CPU 对 Cache 写命中时, 把数据同时写入 Cache

和主存

- ① 写缓冲 (Write Buffer): 为减少全写法直接写入主存的时间消耗, 在 Cache 和主存之间加一个写缓冲
- ② CPU 同时写数据到 Cache 和写缓冲中, 写缓冲再将内容写入主存
- ③ 写缓冲是一个 FIFO 队列, 可以解决速度不匹配的问题, 但若出现频繁写时, 会是写缓冲饱和溢出
- 回写法 (写回法, Write Back): 当 CPU 对 Cache 写命中时, 只把数据写入 Cache, 只有当此块被替换出时才写回主存
 - ① 修改位 (脏位): 为了减少写回主存的次数, 若为 1, 则说明对应的 Cache 行中的块被修改过, 替换时须写回主存

2. Cache 写操作不命中

- 写分配法 (Write Allocate): 把主存块调入 Cache, 之后只修改 Cache, 搭配 Write Back, 标记 Dirty
- 非写分配法 (Not-Write Allocate): 只更新主存单元, 而不把主存块调入 Cache, 搭配 Write Through

3. 现代计算机使用 Write Back + Write Allocate

4 指令系统

1. ISA (指令集体系结构) 规定的内容主要包括

- 指令格式、指令寻址方式、操作类型以及每种操作对应的操作数的相应规定
- 操作数的类型、操作数寻址方式以及是按大端方式还是小端方式存放
- 程序可访问的寄存器编号、个数和位数
- 存储空间的大小和编址方式
- 指令执行过程的控制方式等, 包括程序计数器、条件码定义

2. 指令字长: 一条指令所包含的二进制代码的位数, 取决于操作码的长度、地址码的长度和地址码的个数。指令字长通常为字节的整数倍

- 单字长指令: 指令字长等于机器字长的指令。访存 1 次就能将指令完整取出
- 半字长指令: 指令字长等于半个机器字长的指令
- 双字长指令: 指令字长等于两个机器字长的指令。需访存 2 次才能将指令完整取出

3. 扩展操作码: 使操作码的长度随地址码的减少而增加, 不同地址数的指令可具有不同长度的操作码

- 设地址长度为 n , 上一层留出 m 种状态, 下层可扩展出 $m * 2^n$ 种状态

解析

指令字长 16bit, 4 位为基本操作码

① 13 条三地址指令 $\Rightarrow$ 留出 $2^4 - 13 = 3$ 种

② 40 条二地址指令 $\Rightarrow$ 留出 $3 * 2^4 - 12 = 8$ 种

- 不允许短码是长码的前缀
- 各指令的操作码一定不能重复

4.1 指令的寻址方式

寻址方式：寻找指令或操作数有效地址的方式，即确定本条指令的数据地址及下一条待执行指令的地址的方法。分为**指令寻址**和**数据寻址**两大类

4.1.1 指令寻址

1. 指令寻址：寻找下一条将要执行的指令地址
2. 顺序寻址：通过程序计数器 PC 加 1 (**1 条指令的长度**)，自动形成下一条指令的地址
3. 跳跃寻址：由本条指令给出下条指令地址的计算方式。通过转移类指令实现
 - 绝对转移：地址码直接指出转移目标地址
 - 相对转移：地址码指出转移目的地址相对于当前 PC 值的偏移量

4.1.2 数据寻址

1. 数据寻址：如何在指令中表示一个操作数的地址，或怎样计算出操作数的地址
2. 寻址特征：指明属于哪种寻址方式
3. 形式地址 (A)：指令中的地址码字段并不代表操作数的真实地址
4. 有效地址 (EA)：形式地址结合寻址方式，可以计算出操作数在存储器中的真实地址，即 EA
5. 指令 = 操作码 + 地址码（操作数字段），地址码中可能包含寻址特征位和形式地址
6. 按字节编址，采用大端方式，操作数的机器数为 $ABCD00FFH$
 - ABH 对应一个字节，可以表示一个地址
 - 操作数占 $32bit = 4Byte$ ，即 4 个连续地址
 - 大端 $\Rightarrow$ 在内存中 FFH 表示的地址等于 ABH 的地址加 3
7. **内存地址是无符号数**

解析

操作数采用基址寻址方式。基址寄存器的内容为 $F0000000H$ ，形式地址（补码表示）为 $FF12H$
内存地址无符号数，两者不能直接相加，故需要先转化为真值之后在进行加减

	含义	有效地址	优点	缺点	备注
隐含寻址	不明显地给出操作数的地址，而是隐含操作数的地址		有利于缩短指令字长	需要增加存储操作数或隐含地址的硬件	
立即（数）寻址	指令字中的地址字段指出的是操作数本身，也称 立即数 ，采用补码表示	A 即是操作数	指令在执行阶段不访存，指令执行速度最快	A 的位数限制了立即数的范围	特征：#
直接寻址	指令字中的形式地址 (A) 就是操作数的真实地址 (EA)	$EA = A$	简单，指令在执行阶段仅需访存一次	A 的位数限制了该指令操作数的寻址范围，操作数的地址不易修改	
间接寻址	指令的地址字段是操作数有效地址所在主存单元的地址，即操作数地址的地址	$EA = (A)$	可扩大寻址范围，便于编制程序（用间接寻址可方便地完成子程序返回）	指令执行阶段要多次访存（ n 次间接寻址需 $n + 1$ 次访存）	
寄存器寻址	指令字中的地址字段给出的是操作数所在寄存器的编号	$EA = R_i$	执行阶段不用访存，执行速度快；寄存器数量远小于内存单元数，所以地址码位数较少，指令字长较短	CPU 的寄存器数量有限，寻址能力有限	
寄存器间接寻址	指令字中的 R_i 所指寄存器给出操作数所在主存单元的地址	$EA = (R_i)$	扩大了寻址范围，减少了访存次数		
相对寻址	将 PC 的内容加上指令格式中的形式地址 (A) 而形成操作数的有效地址，其中 (A) 是相对于下一条 PC 值的偏移量，补码表示	$EA = (PC) + A$	操作数的地址随 PC 值的变化而变化，且与指令地址之间总是相差一个固定的偏移量，因此便于程序浮动（一段代码在程序内部的浮动）		广泛应用于转移指令
基址寻址	将基址寄存器 (BR) 的内容加上指令字中的形式地址 (A) 而形成操作数的有效地址	$EA = (BR) + A$	扩大寻址范围（基址寄存器的位数大于形式地址 (A) 的位数）；有利于多道程序设计，并可用于编制浮动程序（整段代码在内存中浮动）		面向系统，用于为多道程序或数据分配存储空间
变址寻址	将变址寄存器 (IX) 的内容加上指令字中的形式地址 (A) 而形成操作数的有效地址	$EA = (IX) + A$	扩大寻址范围；适合编制循环程序		立足于用户，主要用于处理数组问题，变址寄存器的内容有用户设定。 地址 = $a[0]$ 地址 + $sizeof(<TYPE>) \times IX$
堆栈寻址					

4.2 程序的机器级代码表示

通用寄存器						16bit	32bit	说明
31	16	15	8	7	0	AX	EAX	累加器 (Accumulator)
						BX	EBX	基地址寄存器 (Base Register)
						CX	ECX	计数寄存器 (Count Register)
						DX	EDX	数据寄存器 (Data Register)
							ESI	变址寄存器 (Index Register)
							EDI	
							EBP	堆栈基指针 (Base Pointer)
							ESP	堆栈顶指针 (Stack Pointer)

4.2.1 常用指令

1. X86 指令集不允许两个操作数同时来自主存
2. **push** 指令：将操作数压入内存的栈，常用于函数调用
 - X86 默认以 4B 为单位 $\Rightarrow$ 栈中元素固定为 32bit
 - 栈增长方向与内存地址增长方向相反。栈从高地址向低地址增长。即每压入一个元素，ESP 都减小
 - ESP 是栈顶，入栈前先将 ESP 值减 4，然后将操作数压入 ESP 指示的地址
3. **pop** 指令：出栈。出栈前先将 ESP 指示的地址中的内容出栈，然后将 ESP 的值加 4
- 4.

```
sub esp, 12 # 栈顶指针 - 12
add esp, 8 # 栈顶指针 + 8
```

5. **imul** 指令：有符号整数乘法指令。第一个操作数必须为寄存器
6. **idiv** 指令：有符号证书除法指令，只有一个操作数，即除数，被除数是 edx:eax 中的内容 (共 64bit)，操作结果有两部分：商和余数，商送到 eax，余数则送到 edx
7. **xor** 指令：逻辑异或操作指令 (exclusive or)
8. **shl** 指令：逻辑左移指令 (shift left)
9. **shr** 指令：逻辑右移指令 (shift right)
10. **jcondition** 指令：条件转移指令，依据 CPU 状态字中的一系列条件状态转移

```
je <label> # jump when equal
jz <label> # jump when last result was zero
jne <label> # jump when not equal
jg <label> # jump when greater than
jge <label> # jump when greater than or equal to
jl <label> # jump when less than
jle <label> # jump when less than or equal to
```

11. **cmp 指令**：用于比较两个操作数的值，功能相当于 **sub** 指令，不保存操作结果，进根据运算结果设置 CPU 状态字中的条件码，与 **jcondition** 指令搭配使用
12. **test 指令**：对两个操作数进行逐位与运算，功能相当于 **and** 指令，不保存操作结果，进根据运算结果设置 CPU 状态字中的条件码，与 **jcondition** 指令搭配使用
13. **call 指令**：将下一条指令的地址（返回地址）入栈（压栈保存，在函数的栈帧顶部），然后无条件转移到由标签指示的指令，返回值放置在 **EAX** 中
14. **ret 指令**：实现子程序的返回机制，弹出栈中保存的指令地址，然后无条件转移到保存的指令地址
15. **enter 指令**：等价于

```
push ebp # 保存上一层函数的栈帧基址，即EBP旧值
mov ebp, esp # 设置当前函数的栈帧基址，即EBP新值
```

16. **leave 指令**：等价于

```
mov esp, ebp
pop ebp
```

17. 栈帧

- 栈帧最底部 $\Rightarrow$ 上一层栈帧地址 $\Rightarrow$ **EBP** 在上一层栈帧中的旧值
- **EBP**
- 局部变量：C 语言中越靠前定义的局部变量越靠近栈顶
- GCC 为保证数据的严格对齐而规定每个函数的栈帧大小必须是 $16B$ 的倍数
- 调用参数：参数列表中越靠前的参数越靠近栈顶。可以通过 **$R[ebp] + 8$** 或 **$R[ebp] + 12$** 获取实参
- 栈帧最顶部 $\Rightarrow$ 返回地址
- **ESP**

解析

```
804846a 39 c2 cmp dword ptr edx, eax
804846c 7e 0d jle xxxxxxxx
```

读出的信息：

18.
 - **39 c2** $\Rightarrow$ 每条指令占 $2B$
 - **$804846c - 804846a = 2$** 等于第一条指令长度 $\Rightarrow$ 按字节编址
 - **jle** 偏移量为 $0dH$
 - **jle** 采用 PC (EIP) 相对寻址，若跳转，则目标地址 = 下一条指令地址 + 偏移量 = $804846C + 2 + 0DH = 804847BH$

4.3 CISC & RISC

1. 复杂指令系统计算机 (CISC, Complex Instruction Set Computer)：增强原有指令的功能，设置更为复杂的新指令实现软件功能的硬化

2. 精简指令系统计算机 (RISC, Reduce Instruction Set Computer)：减少指令种类和简化指令功能，提高指令的执行速度

	CISC	RISC
指令系统	复杂、庞大	简单、精简
指令数目	一般大于 200 条	一般小于 100 条
指令字长	不固定	定长
可访存指令	不加限制	只有 LOAD/STORE 指令
各种指令执行时间	相差较大	绝大多数在一个周期内完成
各种指令使用频度	相差较大	都比较常用
通用寄存器数量	较少	多
目标代码	难以用优化编译生成搞笑的目标代码程序	采用优化的编译程序，生成代码较为高效
控制方式	绝大多数以微程序控制	绝大多数为组合逻辑控制，效率较微程序高
指令流水线	可以通过一定方式实现	必须实现

5 总线

1. 总线：一组能为多个部件分时和共享的公共信息传送线路

- 分时：同一时刻只允许有一个部件向总线发送信息
- 共享：总线上可以挂接多个部件，多个部件可同时从总线上接受相同的信息

2. 按功能层次分类

- 片内总线：芯片内部的总线
- 系统总线：计算机系统内公共能部件之间相互连接的总线。按系统总线传输信息内容的不同
 - ① 数据总线 (Data Bus, DB)：在各部件之间传输数据、指令和中断类型号等。双向传输总线。数据总线的位数反映一次能传输的数据的位数
 - ② 地址总线 (Address Bus, AB)：指出数据总线上源数据或目的数据所在的主存单元或 I/O 端口的地址。单向传输总线。地址总线的位数反映最大的寻址空间
 - ③ 控制总线 (Control Bus, CB)：用来传输各种命令、反馈和定时信号。一根控制线传输一个信号
 - ④ 数据通路：各个功能部件通过数据总线连接形成的数据传输路径。表示数据流经的路径
- I/O 总线
- 通信总线 (外部总线)：计算机系统之间或计算机系统与其他系统之间传输信息的总线

3. 按数据传输方式分类

- 串行总线：只有一条双向传输或两条单向传输的数据线，数据按比特串行顺序传输，效率低于并行总线。适合长距离通信
- 并行总线：有多条双向传输的数据线，可以实现多比特位的同时传输。若工作频率较高，可能引发数据线之间相互干扰而造成传输错误。适合近距离通信
- 总线带宽 = 总线工作频率 * 总线宽度 $\Rightarrow$ 并行总线并不一定总比串行总线快

4. 总线复用：不同信号在同一条信号线上分式传输的方式

5. 3 通道存储器 (3-channel memory)：计算机中有 3 条独立的数据通道，因此理论带宽约为单通道的 3 倍

5.1 系统总线的结构

1. 单总线结构：将 CPU、主存、I/O 设备都挂载一组总线上
2. 双总线结构：一条主存总线（用于在 CPU、主存和通道之间传送数据），一条 I/O 总线（用于在多个外部设备与通道之间传送数据）
3. 三总线结构：主存总线（用于 CPU 和内存之间传送地址、数据和控制信息）、I/O 总线和直接内存访问 (DMA) 总线
 - 主存总线：CPU 和内存之间传送地址、数据和控制信息
 - I/O 总线：在 CPU 和各类外设之间通信
 - 直接内存访问 (DMA) 总线：在内存和高速外设之间直接传送数据

5.2 总线的性能指标

1. 总线时钟周期：机器的时钟周期
2. 总线时钟频率：机器的时钟频率。是时钟周期的倒数，表示 1 秒内有多少时钟周期
3. 总线传输周期（总线周期）：一次总线操作所需的时间，包括申请、寻址、传输和结束阶段
4. ★总线工作频率：总线上各种操作的频率，是总线周期的倒数，表示 1 秒内传送几次数据
 - 总线周期 = N 个时钟周期 $\Rightarrow$ 总线工作频率 = $\frac{\text{时钟频率}}{N}$
 - 若一个时钟周期传送 K 次数据 $\Rightarrow$ 总线工作频率 = K * 总线时钟频率
5. ★总线宽度（总线位宽）：总线上能够同时传输的数据位数，表示数据总线的根数。e.g. 32 根称为 32 位总线
6. ★总线带宽：单位时间内总线上最多可传输数据的位数，表示每秒传输信息的字节数，单位可用 B/s，指总线本身所能达到的最高传输速率
 - 总线带宽 = 总线工作频率 * (总线宽度/8)

解析

总线工作频率为 22MHz，总线宽度为 16 位 $\Rightarrow$ 总线带宽 = $22M * (16/8) = 44MB/s = 352Mbit/s$

7. 总线复用：一种信号线在不同的时间传输不同的信息
8. 信号线数：地址总线、数据总线和控制总线 3 中总线数的总和

5.3 总线事务

1. 总线事务：在总线上，主设备（如 CPU、DMA 控制器）与从设备（如主存、I/O 设备）之间完成一次完整的信息交换过程
2. 典型的总线事务
 - 存储器读：从主存取数据至处理器
 - 存储器写：向主存写入数据
 - I/O 读写
 - 中断响应
3. 总线事务的阶段
 - 地址传送阶段
 - 从设备响应阶段（数据准备阶段）
 - 数据传送阶段
4. 数据传输方式
 - 非突发传输：每次仅传输一个数据单元，即一个总线宽度的数据。每次都要传送地址
 - 突发传输：事务开始时，主设备发送数据块的首地址，在不释放总线的前提下，连续传送多个数据单元。突发长度：
 - ① 默认固定
 - ② 由主设备确定

5.4 总线定时

总线定时：指总线上主设备与从设备在交换数据时，用于协调双方操作时序的控制协议

	含义	优点	缺点	适用场景
同步定时方式	采用统一的时钟信号来协调发送方和接收方的传送定时关系	传送速度快，具有脚感的传输速率，总线控制逻辑简单	主从强制性同步，缺乏应答或握手机制，可靠性差	总线长度较短且所连接各部件的存取时间比较接近的系统
异步定时方式	通过主从设备之间的握手信号实现定时控制。主设备发出“请求”信号，从设备准备就绪后，发出“回答”信号	总线周期长度可变，能可靠连接工作速度差异较大的设备，自适应性较强	控制逻辑复杂，因多次信号交互，整体传输速度较低	连接速度差异大，对可靠性要求较高而对速率要求不苛刻的系统
半同步定时方式	地址、命令、数据的发送严格参照系统时钟前沿（如上升沿），接收方通常在时钟后沿（如下降沿）进行识别，同时增加一条 Wait 信号线，允许慢速从设备反馈准备状态	在同一时钟下工作，控制比异步方式简单，可靠性较高	系统时钟频率受限于最慢设备，整体速度不高	包含多种速度差异较大设备、但性能要求不高的简单系统
分离式定时方式	将一个总线事务拆分为两个独立子阶段：请求阶段和应答阶段，两阶段之间释放总线。两个子阶段均为单向信息流，且总线在准备期间可被其他事务使用	显著减少总线空闲等待时间，提高总线利用率	控制逻辑复杂，协议开销大	多主设备竞争激烈且从设备响应延迟较大的高性能系统

5.4.1 异步定时方式

根据“请求”和“回答”信号的撤销是否互锁，分为

1. 不互锁方式：主设备发出“请求”后，在预设时延 t_1 后自行撤销；从设备收到请求后立即发出“回答”，并在时延 t_2 后自动撤销。双方无互锁关系，**速度最快，可靠性最差**
2. 半互锁方式：主设备必须收到“回答”后才撤销“请求”（存在互锁）；但从设备发出“回答”后，无须确认“请求”是否撤销，在时延 t_2 后自动撤销（无互锁）
3. 全互锁方式：主设备需收到“回答”才撤销“请求”；从设备需确认“请求”已撤销后才撤销“回答”。双方完全互锁，**可靠性最好，速度最慢**

6 输入/输出系统 & 管理

1. I/O 系统层次结构：CPU → 通道 → 设备控制器 → 设备
 - 在系统重，可能存在多个通道，每个通道可以连接多个控制器，每个控制器可以连接多个物理设备
 - 通道负责控制多个设备控制器，减轻 CPU 的 I/O 负担
 - 设备控制器负责控制设备并于 CPU 通信
 - 设备负责执行 I/O
2. 设备绝对号：系统为每个物理设备分配的唯一编号（物理设备号），用于标识具体的硬件设备
3. I/O 设备按信息交换的单位分类：
 - 块设备：信息交换以数据块为单位。基本特征是传输速率较高、可寻址
 - 字符设备：信息交换以字符为单位。基本特征是传输速率低、不可寻址，并且通常采用中断 I/O 方式
4. I/O 设备按共享属性分类：
 - 独占设备：同一时刻只能由一个进程占用的设备。通常采用静态分配方式
 - 共享设备：**同一时间段**内允许多个进程同时访问的设备。通常采用动态分配方式
 - ① 宏观上同时共享，微观上分时占用
 - ② 通过分时的方式共享使用
 - ③ 必须是可寻址的和可随机访问的设备，从而保证数据的完整性和一致性，提高设备的利用率
 - 虚拟设备：通过 SPOOLing 技术将独占设备改造为共享设备，将一个物理设备变为多个逻辑设备

6.1 I/O 接口

1. I/O 接口（I/O 控制器）：主机和外设之间的交界界面，通过接口可以实现主机和外设之间的信息交换
2. 磁盘驱动器由磁头、磁盘和读/写电路组成，故不属于 I/O 接口

6.1.1 I/O 接口的基本结构

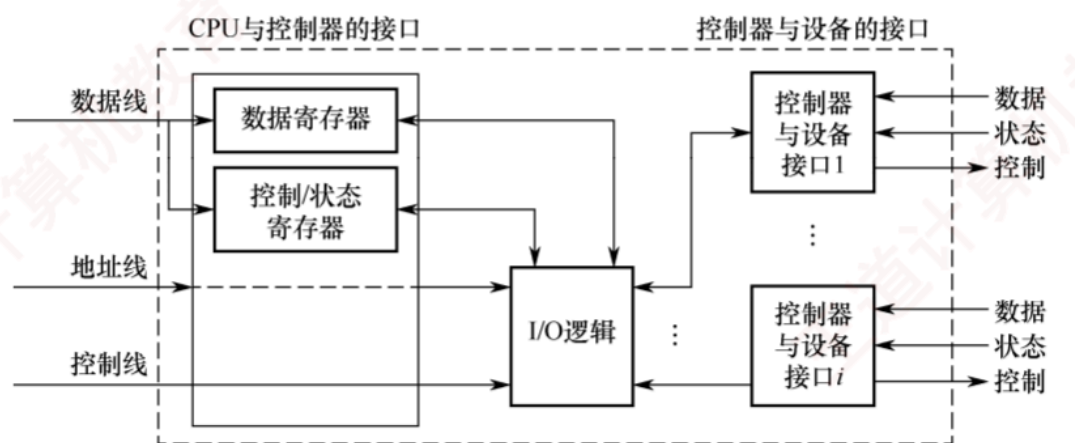


图 5.1 设备控制器的组成

1. 设备控制器与 CPU 的接口：用于实现 CPU 与设备控制器之间的通信
 - 数据线：传送读/写数据、状态信息、控制信息和中断类型号
 - 地址线：传送要访问 I/O 接口中的寄存器的地址
 - 控制线：传送读/写控制信号、中断请求和响应信号、握手信号
2. I/O 逻辑：用于实现对设备的控制
3. 设备控制器与设备的接口
4. I/O 端口：I/O 接口电路中可被 CPU 直接访问的**寄存器**，主要有数据端口（读/写操作）、状态端口（读操作）和控制端口（写操作）
 - 数据缓冲寄存器：暂存与 CPU 或内存之间传送的数据信息
 - 状态寄存器：记录接口和设备的状态信息
 - 控制寄存器：保存 CPU 对外设的控制信息
 - 状态寄存器和控制寄存器在传送方向上是相反的，在访问时间上也是错开的，故合二为一
 - I/O 端口要想被 CPU 访问，就必须要对各个端口进行编址，每个端口对应一个端口地址，编址方式：
 - ① 独立编址（I/O 映射方式）：对所有的 I/O 端口单独进行编址。I/O 端口的地址空间与主存地址空间是两个独立的地址空间。需要设置专门的 I/O 指令来表明访问的是 I/O 地址空间
 - ② 统一编制（存储器映射方式、内存映射 I/O）：把主存地址空间分出一部分给 I/O 端口进行编址，根据地址范围就能区分访问的是 I/O 端口还是主存单元。使用指令系统中的**访存指令**来完成输入/输出操作
5. I/O 指令：对数据缓冲寄存器、状态/控制寄存器的访问操作是通过 I/O 指令完成的，I/O 指令只能在操作系统内核的底层 I/O 软件中使用，是一种特权指令

6.1.2 I/O 接口的功能

1. 识别和接收命令
2. 数据交换

3. 标识和报告设备的状态
4. 地址识别
5. 数据缓冲：接口需设置数据缓冲寄存器，用于数据的暂存，以避免因速度不一致而丢失数据
6. 差错控制

6.2 I/O 控制方式

在输入/输出系统实现主机与 I/O 设备之间的数据传送可以采用不同的控制方式：

1. 程序查询方式（程序直接控制方式、程序轮询方式）
2. 程序中断方式（中断驱动方式）
3. DMA 方式（Direct Memory Access, 直接存储器存取）
4. 通道方式：在系统中设有通道控制部件，每个通道都挂接若干外设，主机在执行 I/O 命令时，只需启动有关通道，通道将执行通道程序，从而完成 I/O 操作

6.2.1 程序查询方式（程序直接控制方式、程序轮询方式）

设置一个数据缓冲寄存器和一个设备状态寄存器，主机进行 I/O 操作时，先读取设备的状态并根据设备状态决定下一步操作。根据 CPU 查询设备状态的方式不同，程序查询分为：

1. 独占查询：一旦设备启动，CPU 就一直持续查询接口状态，可能会引发踏步等待现象
 2. 定时查询：CPU 周期性地查询接口状态，每次总是等到条件满足才进行一个数据的传输，传送完成后返回用户程序。需要保证数据不丢失
1. CPU 负责数据的传送，每完成一次数据传送后，将主存地址加 1，计数值减 1
 2. 数据传送单位是字，每次读/写一个字
 3. CPU 和 I/O 设备只能串行工作

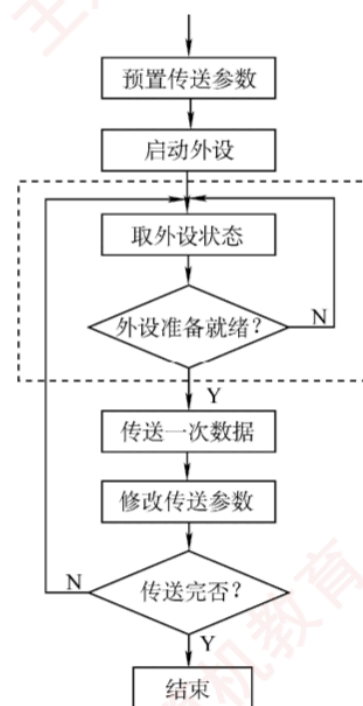


图 7.2 程序查询方式的工作流程

6.2.2 程序中断（中断驱动方式）

1. 程序中断：在计算机执行程序的过程中，出现某些继续处理的异常情况或特殊请求，CPU 暂时中止现行政程序，而转去对这些异常情况或特殊请求进行处理，处理完毕后再返回到原程序的断点处，继续执行原程序
2. CPU 与外设并行工作，传送与主程序串行工作
3. CPU 会在每个指令周期的末尾检查中断
4. 数据传送单位是字，每次读/写一个字

- Cache 缺失：Cache 完全是由硬件实现的，不会涉及中断层面
- 虚拟存储器时效：如缺页 $\Rightarrow$ 缺页中断（内中断）
- 浮点数运算下溢：当做机器零处理，不会引发中断
- 浮点数运算上溢：引发浮点溢出异常（内中断）
- CPU 在执行一条指令的过程中检测异常事件

5. 若准备数据的时间小于中断响应和中断处理的时间，则数据被刷新，造成丢失

6. 中断请求

- 中断源：请求 CPU 中断的设备或事件
- 中断请求标记触发器：记录中断时间并区分不同的中断源，其状态为 1 时，表示该中断源有请求
- 中断请求标记寄存器：由多个中断请求标记触发器组成

7. 中断响应判优：CPU 响应中断请求的先后顺序，通常通过硬件排队器或终端查询程序实现

- 中断优先级包括：
 - ① 响应优先级：由硬件线路或查询程序的查询顺序决定，不可动态改变
 - ② 处理优先级：可利用中断屏蔽技术动态调整，以实现多重中断
- 可屏蔽中断：通过 INTR 线发出，优先级最低，在关中断模式下不被响应
- 不可屏蔽中断：通过 NMI 线发出，用于处理紧急和重要的事件，优先级最高
- 不可屏蔽中断 > 可屏蔽中断
- 高速设备 > 低速设备
- 输入设备 > 输出设备
- 实时设备 > 普通设备

8. CPU 响应中断的条件

- 中断源有中断请求
- CPU 允许中断及开中断（异常和不可屏蔽中断不受此限制）
- 一条指令执行完毕（异常不受此限制），且没有更紧迫的任务

9. 中断隐指令：CPU 响应中断后，经过某些操作，转去执行中断服务程序，这些操作是有硬件直接实现的，即中断隐指令，其本质是 CPU 内部硬件控制逻辑自动完成的一系列操作，所完成的操作：

- 关中断：CPU 响应中断后，首先要保护程序的断点和现场信息，在保护断点和现场的过程中，CPU 不能响应更高级中断源的中断请求
- 保存断点：将原程序的断点保存在栈或特定寄存器中
- 引出中断程序：识别中断源，将对应的服务程序入口地址送入程序计数器 PC。识别中断源方法
 - ① 硬件向量法/向量中断
 - ② 软件查询法/非向量中断
- 中断向量：每个中断源都有一个唯一的类型号，每个中断类型号都对应一个中断服务程序，每个中断服务程序都有一个入口地址，即中断向量

- 中断向量地址：中断向量表的地址，也是中断服务程序入口地址的地址
- 中断向量表：系统中的全部中断向量
- 中断向量法：CPU 响应中断后，通过识别中断源获得中断类型号，然后据此计算出对应中断向量的地址，再根据地址从中断向量表中取出中断服务程序的入口地址，并送入程序计数器 PC，以转去执行中断服务程序
- 向量中断：采用中断向量法的中断

10. 中断处理过程

- 前三步由中断隐指令（硬件自动）完成
- 第四步之后都是由中断服务程序完成
- 第五步开中断：允许更高级中断请求得到响应，已实现中断嵌套
- 最后一步：中断返回指令，需要修改 PC 值，并且要将 CPU 中的所有寄存器都恢复到中断前的状态

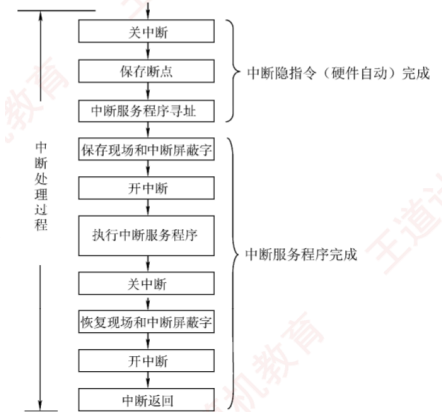


图 7.4 一个支持中断嵌套的典型处理流程

11. 单重中断：在 CPU 执行中断服务程序的过程中，若又出现了新的优先级更高的中断请求，而 CPU 对新的中断请求不予响应
12. 多重中断（中断嵌套）：若 CPU 暂停现行的中断服务程序，转去处理新的中断请求
13. **中断处理优先级**：多重中断的实际优先级处理次序，可以利用中断屏蔽技术动态调整。通过在中断系统中设置**中断屏蔽字**寄存器实现。每个中断源对应中断屏蔽字寄存器中的某一位
 - 1：表示屏蔽该中断源的请求
 - 0：表示可以正常请求
 - 中断屏蔽字：中断屏蔽字寄存器的内容
 - 中断屏蔽字越多 1，优先级越高
 - 每个屏蔽字至少有一个 1 $\Rightarrow$ 屏蔽自身中断
 - 所有中断请求信号都会与中断屏蔽字寄存器中对应位的非值进行逻辑与，然后被送入中断判优电路
 - 通过配置中断屏蔽字，可以动态改变中断处理优先级

6.2.3 DMA 方式（Direct Memory Access, 直接存储器存取）

1. DMA 方式（直接存储器存取方式）：完全由硬件进行成组信息传送的控制方式，具有程序中中断方式的优点（即在数据准备阶段，CPU 与外设并行工作），外设和内存之间开辟了一条直接数据通路，传送信息不再经过 CPU
2. 特点
 - 基本传送单位是数据块，而不再是字
 - 所传送的数据，是从设备直接送入内存的，或者相反，而不再经过 CPU
 - 仅在传送一个或多个数据块的开始和结束时，才需要 CPU 干预
3. 只有具有 DMA 接口的设备才能产生 DMA 请求

4. DMA 优先级高于中断请求

5. ★★CPU 会在每个机器周期（总线事务、存取周期、总线周期、当前流水段长度）结束后检查是否有 DMA 请求以及响应

6. I/O 与主机并行工作，程序与传送并行工作

7. DMA 控制器（DMA 接口）：对数据传送过程中进行控制的硬件

- 当 I/O 设备需要进行数据传送时，（外部设备）向 DMA 控制器发出 DMA 请求信号，通过 DMA 控制器向 CPU 提出 DMA 传送请求，CPU 响应之后将让出系统总线，有 DMA 控制器接管总线进行数据传送 ⇒ DMA 控制器具有控制相同总线的能力

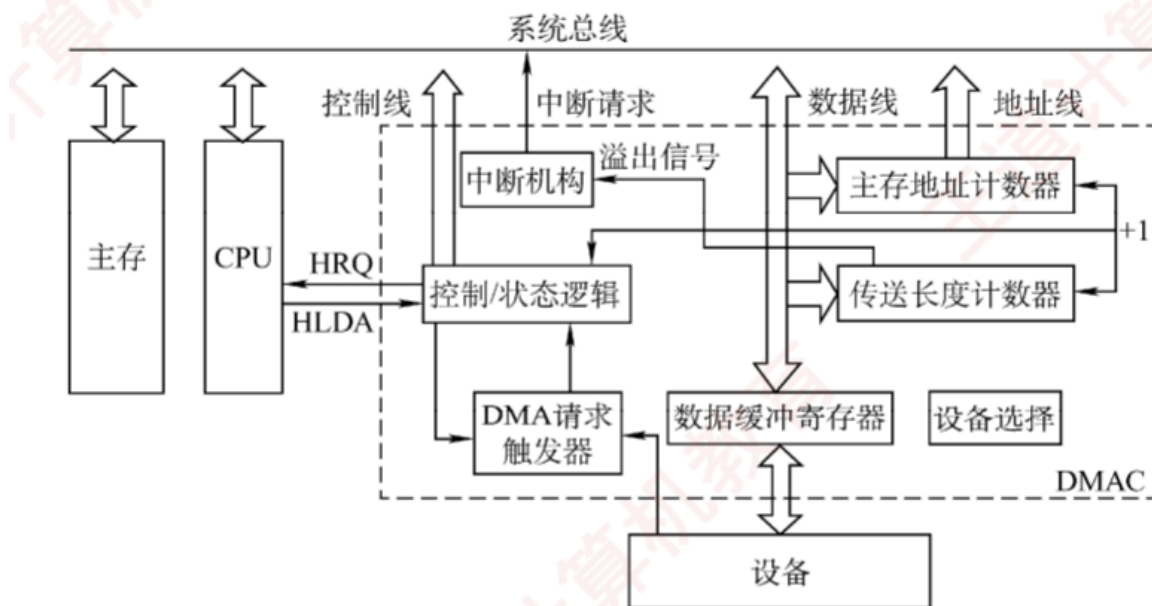


图 7.8 一个简单的 DMA 控制器

• 组成

- ① CR (Command Register, 命令/状态寄存器)：用于存放 CPU 发来的 I/O 命令或设备的状态信息
- ② MAR (Memory Address Register, 主存地址计数器、内存地址寄存器)：存放交换数据的主存地址
- ③ DR (Data Register, 数据寄存器、数据缓冲寄存器)：暂存每次传送的数据。DMA 接口与主存之间传送单位为字
- ④ DC (Data Counter, 传送长度计数器、数据计数器)：记录传送数据的长度
- ⑤ DMA 请求触发器
- ⑥ 控制/状态逻辑：用于指定传送方向，修改传送参数，并对 DMA 请求信号、CPU 响应信号进行协调和同步
- ⑦ 中断机构：当一批数据传送完毕后触发中断机构，向 CPU 提出中断请求

8. DMA 的传送方式：当 I/O 设备和 CPU 同时访问主存时，可能发生冲突，避免冲突方法

• 停止 CPU 访存

- 周期挪用：由于 I/O 访存的有优先级高于 CPU 访存（I/O 不立即访存则可能丢失数据），I/O 设备挪用一個存取周期，传送完一个数据字后立即释放总线。单字传送方式。I/O 设备有 DMA 请求时，有以下情况

- ① CPU 不在访存
- ② CPU 正在访存：等待存取周期结束，CPU 再将总线占有权让出
- ③ 同时请求访存：CPU 要暂时放弃总线占有权
- 交替访存：将 CPU 的工作周期分成两个时间片，一个给 CPU 访存，另一个给 DMA 访存。适用于 CPU 的工作周期比主存存取周期长的情况

9. DMA 传送过程

- 预处理：由 CPU 完成一些必要的准备工作。运行在内核态。由请求 I/O 的进程负责，故请求 I/O 的进程处于运行态
- 初始化 DMA 控制器中的有关寄存器、设置传送方向、测试并启动设备等
- CPU 继续执行原程序，直到 I/O 设备准备好发送的数据（输入情况）或接受的数据（输出情况）时，I/O 设备向 DMA 控制器发送 DMA 请求，再由 DMA 控制器向 CPU 发出总线请求
- 数据传送：DMA（硬件）完成，DMA 以数据块为基本传送单位（CPU 一次交给 DMA 的任务是多少）
- 后处理：DMA 控制器向 CPU 发出中断请求，CPU 执行中断服务程序做 DMA 结束处理。运行在内核态。此时请求 I/O 的进程处于阻塞态

6.2.4 计算

1. 由于端口有限，故必须在外设传输完端口大小的数据时访问端口，以防止数据未被及时读取而丢失
2. 外设准备数据时间 = $\frac{\text{外设准备的数据量}}{\text{数据率}}$
3. CPU 用于外设 I/O 的时间占 CPU 总时间的百分比 = $\frac{\text{每秒 CPU 用于外设 I/O 的时钟周期}}{\text{主频}}$
4. 外设每秒 DMA 次数 = $\frac{\text{数据率}}{\text{每次 DMA 传送块大小}}$

6.3 I/O 软件层次结构

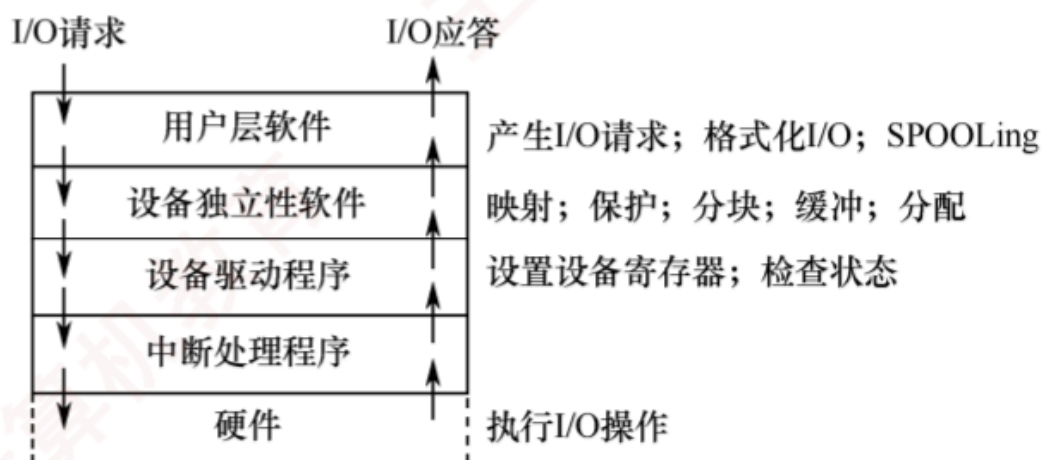


图 5.5 I/O 层次结构

TODO: 区分每层的范围???

1. 用户层软件
2. 设备独立性软件（设备无关性）：实现用户程序与设备驱动器的统一接口、设备命名、设备保护，以及设备的分配与释放等，同时为设备管理和数据传送提供必要的存储空间
 - 让用户程序不用关心具体设备，程序只面对逻辑设备，设备独立性软件负责逻辑设备与物理设备的映射
 - 主要功能：
 - ① 执行所有设备的公有操作
 - ② 向用户层（或文件层）提供统一接口
3. 设备驱动程序：与硬件直接相关，每类设备需要配置一个设备驱动程序
 - 负责具体实现系统对设备发出的操作指令，驱动 I/O 设备工作的驱动程序，是 I/O 进程和设备控制器之间的通信程序
 - 通常以进程的形式存在
4. 中断处理程序：和硬件交互

6.3.1 逻辑设备名 VS 物理设备名

1. 应用程序所用的设备不局限于某个具体的物理设备
2. 在应用程序中，使用逻辑设备名来请求使用某类设备，而在系统实际执行时，必须将逻辑设备名映射成物理设备名
3. 逻辑设备名好处：
 - 增加设备分配的灵活性
 - 易于实现 I/O 重定向（I/O 重定向：用于 I/O 操作的设备可以更换，而不必改变应用程序）
4. 逻辑设备表（Logical Unit Table, LUT）：逻辑设备名与物理设备名的映射
 - $LUT = \text{逻辑设备名} + \text{物理设备名} + \text{设备驱动程序的入口地址}$
 - 两种方式设置 LUT：
 - ① 整个系统重只设置一张 LUT
 - ② 为每个用户设置一张 LUT

6.3.2 设备分配

1. 数据结构
 - 设备控制表（DCT）：每个设备对应一张 DCT，表中的表项就是设备的各个属性
 - 控制器控制表（COCT）：每个设备控制器都对应一张 COCT，操作系统根据 COCT 的信息对控制器进行操作和管理
 - 通道控制表（CHCT）：每个通道对应一张 CHCT，操作系统根据 CHCT 的信息对通道进行操作和管理
 - 系统设备表（SDT）：整个系统只有一张 SDT，记录已连接到系统重的所有物理设备的情况，每个物理设备对应一个表目
2. 设备分配时应考虑的因素
 - 设备的固有属性：独占设备 Or 共享设备 Or 虚拟设备

- 设备的分配算法：FCFS 算法 Or 最高优先级优先算法
- 设备分配中的安全性
 - ① 安全分配方式：每当进程发出 I/O 请求后，便进入阻塞态，知道其 I/O 操作完成时才被唤醒
 - ② 不安全分配方式：进程在发出 I/O 请求后仍继续运行，需要是会继续发出 I/O 请求，仅当进程所请求的设备已被另一进程占用时，才进入阻塞态

3. 步骤

- 分配设备
- 分配控制器
- 分配通道
- 只有设备、控制器和通道都分配成功，设备分配才算成功，之后便可启动设备进行数据传送

6.4 应用程序 I/O 接口

1. 应用程序 I/O 接口是软件和操作系统之间的接口，I/O 接口（I/O 控制器）是 CPU/内存和外部设备之间的硬件接口
2. 分类：
 - 字符设备接口
 - 字符设备：数据的存取和传输是以字符为单位的设备
 - 基本特征：传输速率较低、不可寻址，并且在输入/输出是通常采用**中断驱动方式**
 - 块设备接口
 - ① 块设备：数据的存取和传输是以数据块为单位的设备
 - ② 基本特征：传输速率较高，可寻址。磁盘设备的 I/O 通常采用**DMA 方式**
 - 网络设备接口：网络 I/O 接口为网络套接字接口

3. 模式

- 阻塞 I/O：当用户进程调用 I/O 操作时，进程就被阻塞，并移到阻塞队列，I/O 操作完成后，进程才被唤醒，移到就绪队列
- 非阻塞 I/O：当用户进程调用 I/O 操作是，不阻塞该进程，但进程需要不断询问 I/O 操作是否完成，在 I/O 执行阶段，进程还可以做其他事情

6.5 缓冲管理

1. 磁盘高速缓存技术：利用内存中的存储空间来暂存从磁盘中读出的一系列盘块中的信息。磁盘高速缓存逻辑上属于磁盘，物理上是驻留在内存中的盘块

6.5.1 缓冲区 (Buffer)

1. 缓冲区是一种临界资源，在使用缓冲区时是互斥访问的
2. 引入缓冲区的目的
 - 缓和 CPU 与 I/O 设备间速度不匹配的矛盾

- 减少对 CPU 的中断频率，放宽对 CPU 中断响应时间的限制
- 解决基本数据单元大小不匹配的问题
- 提高 CPU 和 I/O 设备之间的并行性

3. 缓冲区的实现方法

- 采用硬件缓冲器
- 利用内存作为缓冲区

4. 根据系统设置缓冲区的个数，缓冲技术分类：

- T：从设备将一块数据输入缓冲区的时间
- M：操作系统将该缓冲区中的数据传送到工作区的时间
- C：CPU 对这一块数据进行处理的时间
- 单缓冲：每当用户进程发出一个 I/O 请求，操作系统便在内存中为之分配一个缓冲区
 - ① 一个缓冲区的大小就是一个块
 - ② 必须等缓冲区装满后才能从缓冲区取走数据
 - ③ T 和 C 是可以并行的
 - ④ 单缓冲区处理每块数据的平均时间： $\max(C, T) + M$
 - ⑤ 缓冲区是共享资源，故使用时必须互斥
- 双缓冲（缓冲对换）
 - ① C 和 M 是可以与 T 并行的
 - ② 双缓冲区处理每块数据的平均时间： $\max(C + M, T)$
 - ③ 可以实现双向数据传输
 - ④ 若输入和输出的速度相差较大时，效果不理想
- 循环缓冲（多缓冲机制）：包含多个大小相等的缓冲区，多个缓冲区链接称一个循环队列
- 缓冲池：包含一个用于管理自身的数据结构和一组操作函数的管理机制，用于管理多个缓冲区
 - ① 缓冲区按其使用状况分为
 - (a) 空缓冲队列
 - (b) 输入队列
 - (c) 输出队列
 - ② 4 中工作缓冲区
 - (a) hin：用于收容输入数据的工作缓冲区
 - (b) sin：用于提取输入数据的工作缓冲区
 - (c) hout：用于收容输出数据的工作缓冲区
 - (d) sout：用于提取输出数据的工作缓冲区

③ 4 种工作方式

- (a) 收容输入
- (b) 提取输入
- (c) 收容输出
- (d) 提取输出

6.5.2 SPOOLing 技术（假脱机技术）

1. 目的：缓和 CPU 的高速型与 I/O 设备的低速性之间的矛盾，将独占设备改造成共享设备的技术
2. 多道程序技术（Multiprogramming）：让 CPU 和 I/O 设备尽量同时工作，提高系统资源利用率
3. 多道程序技术是 SPOOLing 技术的基础。SPOOLing 利用多道程序技术，使输入进程、输出进程和用户进程并发执行，并借助磁盘作为缓冲区，将独占设备虚拟为共享设备，从而提高 CPU 和设备的利用率
4. SPOOLing 技术是一种以空间换时间的技术
5. SPOOLing 技术：利用专门的外围控制机，先将低速 I/O 设备上的数据传送到高速磁盘上，或者相反，当 CPU 需要输入数据时，便可直接从磁盘中读取数据；反之，当 CPU 需要输出数据时，也能以很快的速度将数据先输出到磁盘上
6. 例子：扫描仪、打印机
7. 组成
 - 输入井和输出井
 - 输入缓冲区和输出缓冲区
 - 输入进程和输出进程
8. 核心程序
 - 预输入程序
 - 缓输出程序
 - 井管理程序